Data-Driven Design & Analysis of Structures & Materials (3dasm)

Instructor: Miguel A. Bessa

Brown University

Homework 6

Deliver a **ZIP folder** **with two files (PDF and Jupyter notebook)** containing the answers to the questions of this assignment. **UPLOAD to CANVAS in the Assignments section (Homework 6) by the due date.**

Due date: until 11:59pm of day announced in course home page.

This Homework aims to help you reason about Gaussian processes: what they predict, noise that is learned and modeled with the staggered scheme of Lecture 13, the choice of kernel and of its hyperparameters, more than one input, and why we scale the data (Lectures 13 to 16), using an AI coding agent to write and run the code. This is a homework containing questions to be solved with **pen and paper** as well as with **code**: the notebook you submit contains your prompts, the code the agents wrote, their results and your answers.

This homework has two required exercises (Exercises 1 and 2) and two optional exercises (Exercises 3 and 4). A full mark (A+) needs only Exercises 1 and 2. The optional exercises cover foundational concepts of the course, the choice of a kernel and of its hyperparameters, and a basis that finds a formula versus a kernel that needs no basis, and we recommend them: they also prepare you for the midterm. Correct answers to the optional exercises award additional credit, up to a top mark of 100% in the homework: each optional exercise answered without significant errors makes up for one significant error, or two minor errors, in the required exercises. Optional exercises never lower your mark.

How to work. Start from the notebook `Assignments/3dasm_Homework6.ipynb`: copy it to `Assignments/your_Assignments/HW6/your_Homework6.ipynb` and work there. As in the lectures, an AI coding agent writes and runs the code, and the notebook is the source of truth: you write the agent's instructions file (`CLAUDE.md`) in a notebook cell, each prompt you give the agent is a notebook cell, and the cells that we provide load each agent's `results.json` from its task folder.

- **Model.** Use Sonnet or Haiku (`set_model("sonnet")` or `set_model("haiku")`); the course's helper does not accept other models.
- **Grading.** The TA runs your notebook **from scratch** (every agent runs again), with the model you declare in it. The regenerated numbers must be the same as the ones you submitted: if they are not, your instructions left something for the agent to guess. Your numbers are also compared with the solution: the homework fixes the dataset, the split and the models, so a precise notebook reproduces them exactly. Your predictions, checks and interpretations are graded as usual; base them on the numbers and figures your notebook stores, not on the agent's summary.
- **Prompts** are written in words. Short expressions that pin the data, the randomness or the method are fine (e.g. `random_state=123`); code that implements the task is not.
- **Your prompts and your instructions file are graded.** In the exercises with the agent, a good prompt defines the problem **uniquely**: the data, the randomness (seeds, and the order of the draws), the model and the method, and what to store and plot. A prompt that leaves choices open makes the outcome depend on what the agent happens to guess, which can change from one run to the next. The TA re-runs your notebook with fresh agents: a number that changes counts as an error, even if your answer was right. The TA also reads your prompts: an ambiguity that happened to reproduce counts as a minor error.
- **Write your prompts in your own words.** A prompt that only points to a question (e.g. "solve exercise 4.2") or that pastes it is not a prompt: that item counts as not answered. You may copy the specifications (numbers, seeds, names of functions).

- Each exercise that says **(pen and paper)** should not be resolved with an AI agent (remember that you won't have access to AI in the midterm, so it's better for you not to use it for these questions)
- Each exercise that says **(with the agent)** is appropriate to resolve with an agent as we introduced in the lectures.
  - Note: **writing your own code instead of using the agent is allowed** for the exercises signalled as **(with the agent)**. Put your code in the same notebook cell (not in a separate file).
- **Before you submit**, delete the task folders and run the notebook from the start, so that its outputs come from one clean run (this is also what the TA will do when grading).
- **Submit one ZIP file** with your notebook (`.ipynb`), which the TA re-runs, and a PDF of that same notebook after your clean run. If you write the pen and paper answers by hand, add a scan of them as a third file. Do not include the task folders (they are not needed: the notebook is the source of truth).

Exercise 1 (pen and paper)

Consider a Gaussian process with zero prior mean and kernel $k(\mathbf{x}, \mathbf{x}')$, trained on N points with outputs $\mathbf{y}$ and the noise variances of the observation distribution at the training points, $\mathbf{\Lambda} = \text{diag}\left(\sigma^2_{y|z}(\mathbf{x}_1), \ldots, \sigma^2_{y|z}(\mathbf{x}_N)\right)$. At a new input $\mathbf{x}^*$, the posterior of the mean of the observation distribution, $\mu_{y|z}(\mathbf{x}^*)$, is Gaussian with mean $\mu^* = \mathbf{k}^{*T}(\mathbf{K}+\mathbf{\Lambda})^{-1}\mathbf{y}$ and variance $\mathbb{V}[\mu_{y|z}(\mathbf{x}^*)] = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}^{*T}(\mathbf{K}+\mathbf{\Lambda})^{-1}\mathbf{k}^*$, where $\mathbf{K}$ is the kernel matrix of the training inputs and $\mathbf{k}^*$ the kernel vector, $k^*_n = k(\mathbf{x}_n, \mathbf{x}^*)$. The PPD of a new measurement y^* has the same mean, and the noise variance at the new input, $\sigma^2_{y|z}(\mathbf{x}^*)$, added to this variance.

1. What a Gaussian process predicts.

   1.1. Show that the PPD mean is a weighted sum of kernels centered at the training points, $\mu^* = \sum_{n=1}^{N} a_n\, k(\mathbf{x}^*, \mathbf{x}_n)$, and give the vector of coefficients $\mathbf{a}$. How many coefficients are there, and what happens to them when you add one training point? Compare with Bayesian linear regression with M basis functions, and explain in which sense a Gaussian process is **non-parametric**.

   1.2. **Noiseless data**: let $\mathbf{\Lambda} = \mathbf{0}$, with $\mathbf{K}$ invertible. Show that at a training input, $\mathbf{x}^* = \mathbf{x}_m$, the PPD has mean y_m and variance zero. (Hint: $\mathbf{k}^*$ is then the m-th column of $\mathbf{K}$, so $\mathbf{K}^{-1}\mathbf{k}^* = \mathbf{e}_m$, the m-th column of the identity matrix.) In practice, scikit-learn adds a small value to the diagonal even for noiseless data (`alpha=1e-10` by default): why?

   1.3. **Far from the data**: consider the kernel $k(x, x') = s^2 \exp\left(-\frac{(x-x')^2}{2l^2}\right)$ in one dimension. What do the PPD mean and variance tend to when x^* moves far from all the training inputs? Interpret both in terms of the prior, and say which kind of uncertainty (aleatoric or epistemic) this variance is.

   1.4. **The White kernel**: let $\mathbf{\Lambda} = \mathbf{0}$ and use the kernel $k_W(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}, \mathbf{x}') + s_2^2\, \delta(\mathbf{x}, \mathbf{x}')$, where $\delta(\mathbf{x}, \mathbf{x}') = 1$ if $\mathbf{x} = \mathbf{x}'$ and 0 otherwise. For a new input $\mathbf{x}^*$ that is not a training input, write $\mathbf{K}_W$, $\mathbf{k}^*_W$ and $k_W(\mathbf{x}^*, \mathbf{x}^*)$ in terms of $\mathbf{K}$, $\mathbf{k}^*$ and $k(\mathbf{x}^*, \mathbf{x}^*)$. Show that the PPD mean is the same as with the kernel k and the noise $\mathbf{\Lambda} = s_2^2 \mathbf{I}_N$ (scikit-learn's `alpha` $= s_2^2$), and that the variance differs by s_2^2. Which of the two variances is the one of a credible interval for $\mu_{y|z}(\mathbf{x}^*)$, and which the one of a posterior predictive interval for a new measurement y^*?

   1.5. **Scaling**: a Gaussian process with zero prior mean and the kernel of 1.3. is trained on outputs that are all close to 840 (as in the multidimensional example of the lectures). Using 1.3., what does it predict away from the training points, and what must the optimizer do to s to fit the data? Write the expressions of scikit-learn's `StandardScaler` and `MinMaxScaler` (with range $[-1, 1]$) for one feature. Standardizing the outputs (`normalize_y=True`) means fitting the Gaussian process to $\tilde{y} = (y - m)/c$: how must its PPD mean and standard deviation be transformed back to the units of y?

Exercise 2 (with the agent)

In this exercise you model a noise that changes with the input, without knowing the truth. The file `Assignments/data/HW6_measurements.csv` has 1000 measurements (x, y), with $x \in [0, 10]$, of a quantity whose mean and noise we do not reveal. Split them into training and test sets with `train_test_split` (`test_size=0.25`, `random_state=123`), and use the test set only to evaluate. The model of the mean is a Gaussian process with the kernel `ConstantKernel(1.0, (1e-3, 1e3)) * RBF(1.0, (1e-2, 1e2))`, `n_restarts_optimizer=2` and `random_state=0`, on the raw x. The model of the noise is $\log \sigma^2_{y|z}(x) = \mathbf{v}^T \boldsymbol{\psi}(x)$, with a basis $\boldsymbol{\psi}$ of the raw x and a Uniform prior on $\mathbf{v}$, fitted with the **staggered scheme** of Lecture 13:

- **start**: fit the Gaussian process (with the same settings) with `WhiteKernel(1.0, (1e-5, 1e2))` added to the kernel and `alpha=1e-10`, and set $\mathbf{v}$ to zero except the weight of the constant, $\log s_2^2$ (the noise level learned by the White kernel); then a **mean step**: train the Gaussian process (without White kernel) with `alpha` $= \sigma^2_{y|z}(x_n) = e^{\mathbf{v}^T \boldsymbol{\psi}(x_n)}$;
- then repeat 10 times: (i) the **noise step**, the point estimate of Lecture 13 with 100 steps of Newton's method from the current $\mathbf{v}$, with $\tilde{r}_n^2 = (y_n - \mu_n^*)^2 + \mathbb{V}[\mu_{y|z}(x_n)]$, where μ_n^* and $\mathbb{V}[\mu_{y|z}(x_n)]$ are the PPD mean and the variance of $\mu_{y|z}$ at the training input x_n (the standard deviation returned by scikit-learn, which excludes `alpha`); (ii) a mean step with the new $\mathbf{v}$.

The final model is the last mean step with the last $\mathbf{v}$, and its posterior predictive interval of y^* adds $\hat{\sigma}^2_{y|z}(x^*)$ to the variance returned by scikit-learn. The **coverage** is the fraction of test points with $|y - \text{PPD mean}| \leq 1.96$ PPD standard deviations, computed in each of the five bins $[0, 2)$, $[2, 4)$, $[4, 6)$, $[6, 8)$ and $[8, 10]$.

2. Learning the noise, and discovering its formula.

   2.1. **Before running anything**, predict which of the three bases of 2.2. will give the best coverage, and why.

   2.2. Fit the model with three bases for the log-variance: (a) $\boldsymbol{\psi}(x) = [1, x]$; (b) $\boldsymbol{\psi}(x) = [1, x, \sin x]$; (c) $\boldsymbol{\psi}(x) = [1, x, \sin x, \cos x, x \sin x, x \cos x]$. For each, report the weights $\hat{\mathbf{v}}$, the fitted kernel, the coverage in each bin and the test MSE of the PPD mean. Plot, in one figure, the estimated noise variance $\hat{\sigma}^2_{y|z}(x)$ of the three models together with the squared residuals $\tilde{r}_n^2$ of model (c) (on a logarithmic scale), and show the final PPD of model (c).

   2.3. Read the weights of model (c): which basis functions matter? Suggest the expression that generated the noise, $\log \sigma^2(x)$. Then fit the model with only the basis functions that you selected, and report its weights and its coverage in each bin. Is it as good as model (c)?

   2.4. **What can you conclude?** Compare with your predictions of 2.1.. What does the basis $\boldsymbol{\psi}$ decide about the model of the noise? Did the model of the noise change the PPD mean, and why? Why could a basis with six functions find the formula from these data?

Exercise 3 (optional, with the agent)

Consider again the dataset without noise of Homework 4: $f(x) = x \sin(x)$ at 40 values of x drawn uniformly at random in $[0, 10]$, in a single draw from NumPy's default random number generator with seed 123, sorted in ascending order, and split into training (80%) and testing (20%) sets with "train_test_split" and its "random_state" set to 123. As in Homework 4, the training sets of size $N = 6$ and 21 are the first N points of the training set, in the order in which "train_test_split" returns them. (If you saved the file `HW4_noiseless_dataset.csv`, it has these points; your prompt must define them anyway, so that the TA can regenerate them.)

3. Gaussian processes on the dataset of Homework 4.

   3.1. **Before running anything**, predict: compared with the polynomials of Homework 4, how well will a Gaussian process predict the testing points with 6 training points? Which of the kernels below do you expect to be best for $x \sin(x)$? What will the Gaussian processes do outside $[0, 10]$?

   3.2. Train Gaussian processes with four kernels, each multiplied by `ConstantKernel(1.0, (1e-2, 1e4))`:

   - `RBF(1.0, (1e-2, 1e2))`;

- `Matern(1.0, (1e-2, 1e2), nu=2.5)`;
- `Matern(1.0, (1e-2, 1e2), nu=1.5)`;
- `ExpSineSquared(1.0, 5.0, length_scale_bounds=(1e-2, 1e2), periodicity_bounds=(1e-1, 1e2))`;

with `alpha=1e-8`, `n_restarts_optimizer=5` and `random_state=0`, for each training set size. Report the testing R^2 and MSE and the fitted kernels, and show one figure for $N = 6$ with the four Gaussian processes (mean and 95% credible interval of $\mu_{y|z}(x^*)$), the training and testing points and f, for $x \in [-2, 12]$. Which hyperparameters ended at one of their bounds? Compare with your best polynomials of Homework 4.

3.3. **Check** the Gaussian process with the RBF kernel and $N = 21$ against Exercise 1: report the largest $|\mu^* - y_n|$ at $x^* = x_n$ and the largest standard deviation at the 21 training inputs (1.2.), and the standard deviation at $x = 30$ together with the fitted s (1.3.).

3.4. **What can you conclude?** Compare with your predictions of 3.1.. Which hyperparameters ended at one of their bounds, and what does that tell you about the kernel? Why does the Exp-Sine-Squared kernel fail with 6 points? Which kernel would you choose for this problem, and how would you choose it if you did not know f?

Exercise 4 (optional, with the agent)

Consider the function of two inputs $f(x_1, x_2) = 2\sin(x_1) + \cos(2x_2) + 0.5\sin(x_1)\cos(x_2)$ in the domain $[0, 2\pi]^2$. Create a noisy dataset: draw the 60 inputs as an array of shape 60×2 (one row per point) uniformly in $[0, 2\pi]$, in a single draw from NumPy's default random number generator with seed 123, and compute $y = f(x_1, x_2) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, 0.2^2)$ (60 values drawn after the inputs, with the same generator). Split it into training (80%) and testing (20%) sets with "train_test_split" and its "random_state" set to 123. Evaluate every model also on a 60×60 grid of $[0, 2\pi]^2$ (`np.linspace(0, 2*np.pi, 60)` in each input), where the **grid** R^2 compares the prediction with f (without noise). For the Lasso models, use scikit-learn's `Lasso` with `max_iter=100000` (and the default intercept), and choose hyperparameters by grid search with 5-fold cross-validation on the training set (`KFold`, points shuffled, `random_state=123`, scored by the MSE).

4. A basis that finds the formula, or a kernel that needs no basis.

4.1. Plot f as a contour plot with the training points on top. **Before running anything else**, write down your predictions: which of the three models below do you expect to predict f best, and how many non-zero weights do you expect each Lasso model to keep?

4.2. **Lasso with two bases.** (a) A **polynomial basis** of the inputs rescaled to $[-1, 1]$ (`MinMaxScaler` fitted on the training set): choose the degree (3, 5 or 7) and α (16 values of `np.logspace(-5, 0, 16)`). (b) A **trigonometric basis** of the raw inputs: the 12 functions $\sin(kx_i)$ and $\cos(kx_i)$ for $k = 1, 2, 3$ and $i = 1, 2$, plus the 16 products of one function of $\{\sin x_1, \cos x_1, \sin 2x_1, \cos 2x_1\}$ with one of $\{\sin x_2, \cos x_2, \sin 2x_2, \cos 2x_2\}$ (28 functions); choose α among the same 16 values. Report, for both, the chosen hyperparameters, the testing MSE, the grid R^2 and the number of non-zero weights, and for (b) the non-zero weights with the function each one multiplies. Show contour plots of both models.

4.3. **Lasso finds the formula.** With the trigonometric basis, fit Lasso for $\alpha = 10^{-3}$, 10^{-2}, 0.05, 0.1 and 0.5, and report the number of non-zero weights and the weights of the three terms of f. For which α does Lasso keep exactly the terms of f? How do their weights compare with the true coefficients, and why (Exercise 1 of Homework 5)? Then fit Linear Least Squares using only the basis functions that Lasso kept, and report its weights, testing MSE and grid R^2.

4.4. **A Gaussian process.** Standardize the inputs (`StandardScaler` fitted on the training set) and train a Gaussian process with the kernel `ConstantKernel(1.0, (1e-3, 1e3)) * RBF([1.0, 1.0], (1e-2, 1e2))` (one length scale per input), the known noise `alpha` $= 0.2^2$, `n_restarts_optimizer=5` and `random_state=0`. Report the fitted kernel, the testing MSE and the grid R^2, and show contour plots of the PPD mean, of its error $|\mu - f|$ and of its standard deviation on the grid, with the training points on top. Where is the standard deviation largest, and does it point to where the error is largest?

4.5. **What can you conclude?** Compare with your predictions of 4.1.. Why does the polynomial basis struggle with 48 training points in two dimensions? What did the Gaussian process need to know about f, and what did the trigonometric basis need to know? Which model would you report to a colleague who wants to understand f, and which to a colleague who wants a prediction with its uncertainty? Can you trust the testing MSE on 12 points to rank these models?